



```
In [ ]: import pandas as pd

df = pd.read_excel("d.csv.xlsx")
df.head(5)
```

```
In [2]: df.shape
```

```
Out[2]: (5500, 4)
```

```
In [ ]:
```

```
In [3]: #Check dataset
print(df.shape)
print(df.columns)
```

```
(5500, 4)
```

```
Index(['datetime', 'patient_id', 'code', 'value'], dtype='object')
```

```
In [5]: import pandas as pd

# =====
# STEP 1: LOAD DATASET
# =====
df = pd.read_excel("d.csv.xlsx")

print("Dataset Loaded!")
print("Shape:", df.shape)

# =====
# STEP 2: APPLY CSP (NEW LOGIC)
# =====
c1 = [] # High Code
c2 = [] # High Value
c3 = [] # Repeated Patient
c4 = [] # Critical Time (night)

risk_scores = []
results = []
reasons = []

# Count patient frequency
patient_counts = df['patient_id'].value_counts()

for index, row in df.iterrows():

    risk = 0
    reason = []

    # ♦ Constraint 1: High code value
    if row['code'] > 60:
        c1.append(1)
        risk += 1
```

```

        reason.append("High Code")
    else:
        c1.append(0)

# ♦ Constraint 2: High measurement value
if row['value'] > 1:
    c2.append(1)
    risk += 1
    reason.append("High Value")
else:
    c2.append(0)

# ♦ Constraint 3: Frequent patient visits
if patient_counts[row['patient_id']] > 5:
    c3.append(1)
    risk += 1
    reason.append("Frequent Visits")
else:
    c3.append(0)

# ♦ Constraint 4: Night time (risk assumption)
hour = row['datetime'].hour
if hour < 6 or hour > 20:
    c4.append(1)
    risk += 1
    reason.append("Odd Time Visit")
else:
    c4.append(0)

# Risk Score
risk_scores.append(risk)

# ♦ Decision
if risk >= 3:
    label = "High Risk"
elif risk == 2:
    label = "Medium Risk"
else:
    label = "Low Risk"

results.append(label)

# Reason
if len(reason) == 0:
    reasons.append("Normal Behavior")
else:
    reasons.append(", ".join(reason))

# =====
# ADD TO DATAFRAME
# =====
df['C1_HighCode'] = c1

```

```

df['C2_HighValue'] = c2
df['C3_FrequentVisit'] = c3
df['C4_OddTime'] = c4

df['Risk_Score'] = risk_scores
df['AI_Result'] = results
df['Reason'] = reasons

# =====
# ANALYSIS
# =====
analysis = []

for res in results:
    if res == "Low Risk":
        analysis.append("Normal")
    elif res == "Medium Risk":
        analysis.append("Monitor")
    elif res == "High Risk":
        analysis.append("Critical")
    else:
        analysis.append("Unknown")

df['Analysis'] = analysis

# =====
# SUGGESTIONS
# =====
suggestions = []

for val in df['Analysis']:
    if val == "Normal":
        suggestions.append("No Action")
    elif val == "Monitor":
        suggestions.append("Check Patient")
    elif val == "Critical":
        suggestions.append("Immediate Attention")
    else:
        suggestions.append("Unknown")

df['Suggestions'] = suggestions

# =====
# SHOW OUTPUT
# =====
print("\n===== FINAL TABLE =====")
print(df.head(10))

# =====

```

```
# DISTRIBUTION
# =====
print("\n===== RISK DISTRIBUTION =====")
print(df['AI_Result'].value_counts())

# =====
# SAVE
# =====
df.to_csv("final_output.csv", index=False)
print("\nOutput saved as final_output.csv")
```

Dataset Loaded!
Shape: (5500, 4)

===== FINAL TABLE =====

	datetime	patient_id	code	value	C1_HighCode	C2_HighValue	\
0	1988-03-27 08:00:00	68	58	1	0	0	
1	1988-03-27 08:00:00	68	34	0	0	0	
2	1988-03-27 12:00:00	68	60	1	0	0	
3	1988-03-27 18:00:00	68	62	2	1	1	
4	1988-03-28 08:00:00	68	58	1	0	0	
5	1988-03-28 08:00:00	68	34	0	0	0	
6	1988-03-28 12:00:00	68	60	1	0	0	
7	1988-03-28 18:00:00	68	62	1	1	0	
8	1988-03-31 08:00:00	68	58	0	0	0	
9	1988-03-31 08:00:00	68	34	0	0	0	

	C3_FrequentVisit	C4_OddTime	Risk_Score	AI_Result	\
0	1	0	1	Low Risk	
1	1	0	1	Low Risk	
2	1	0	1	Low Risk	
3	1	0	3	High Risk	
4	1	0	1	Low Risk	
5	1	0	1	Low Risk	
6	1	0	1	Low Risk	
7	1	0	2	Medium Risk	
8	1	0	1	Low Risk	
9	1	0	1	Low Risk	

	Reason	Analysis	Suggestions
0	Frequent Visits	Normal	No Action
1	Frequent Visits	Normal	No Action
2	Frequent Visits	Normal	No Action
3	High Code, High Value, Frequent Visits	Critical	Immediate Attention
4	Frequent Visits	Normal	No Action
5	Frequent Visits	Normal	No Action
6	Frequent Visits	Normal	No Action
7	High Code, Frequent Visits	Monitor	Check Patient
8	Frequent Visits	Normal	No Action
9	Frequent Visits	Normal	No Action

===== RISK DISTRIBUTION =====

Low Risk 3663
Medium Risk 1369
High Risk 468
Name: AI_Result, dtype: int64

Output saved as final_output.csv

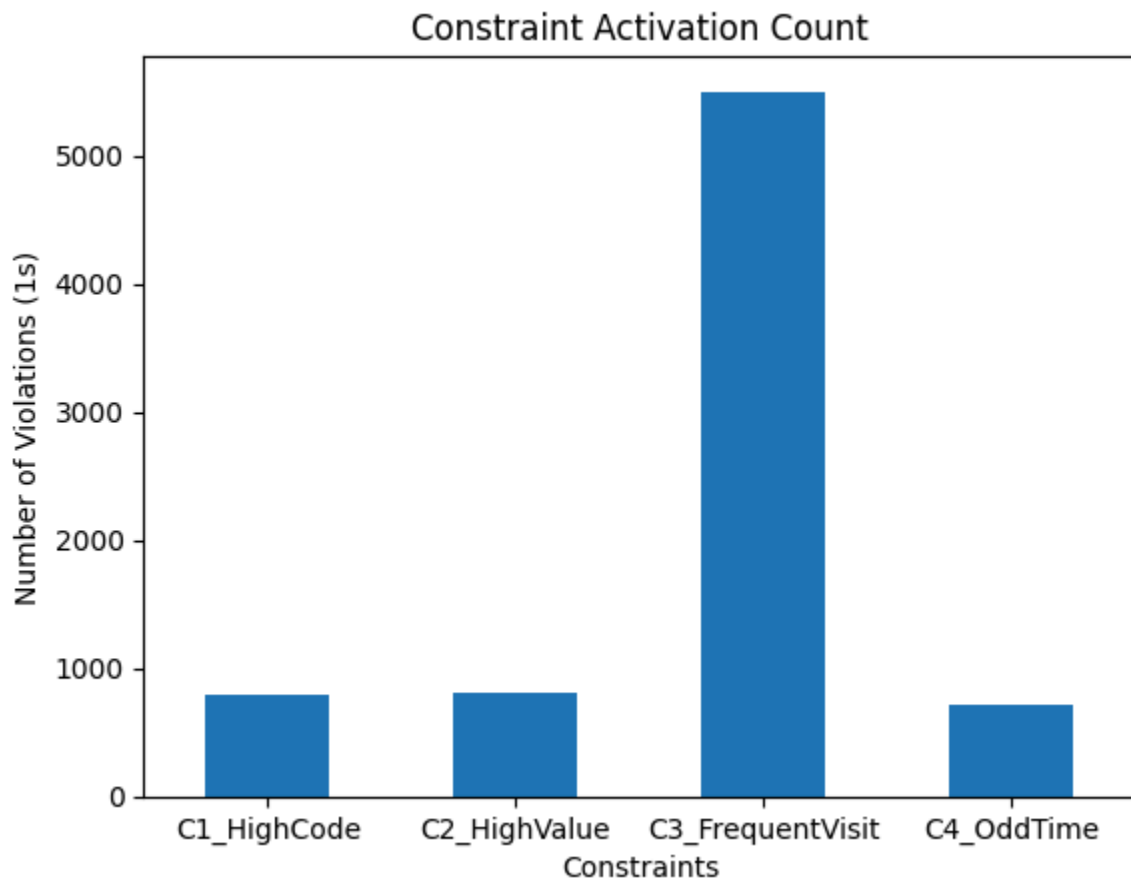
```
In [6]: # =====  
# CONSTRAINT ACTIVATION (UPDATED)  
# =====  
import matplotlib.pyplot as plt  
  
constraint_counts = df[['C1_HighCode', 'C2_HighValue', 'C3_FrequentVisit', 'C4_Oc
```

```

constraint_counts.plot(kind='bar')

plt.title("Constraint Activation Count")
plt.xlabel("Constraints")
plt.ylabel("Number of Violations (1s)")
plt.xticks(rotation=0)
plt.show()

```



```

In [7]: # =====
# RISK DISTRIBUTION (UPDATED)
# =====
import matplotlib.pyplot as plt

risk_counts = df['AI_Result'].value_counts()

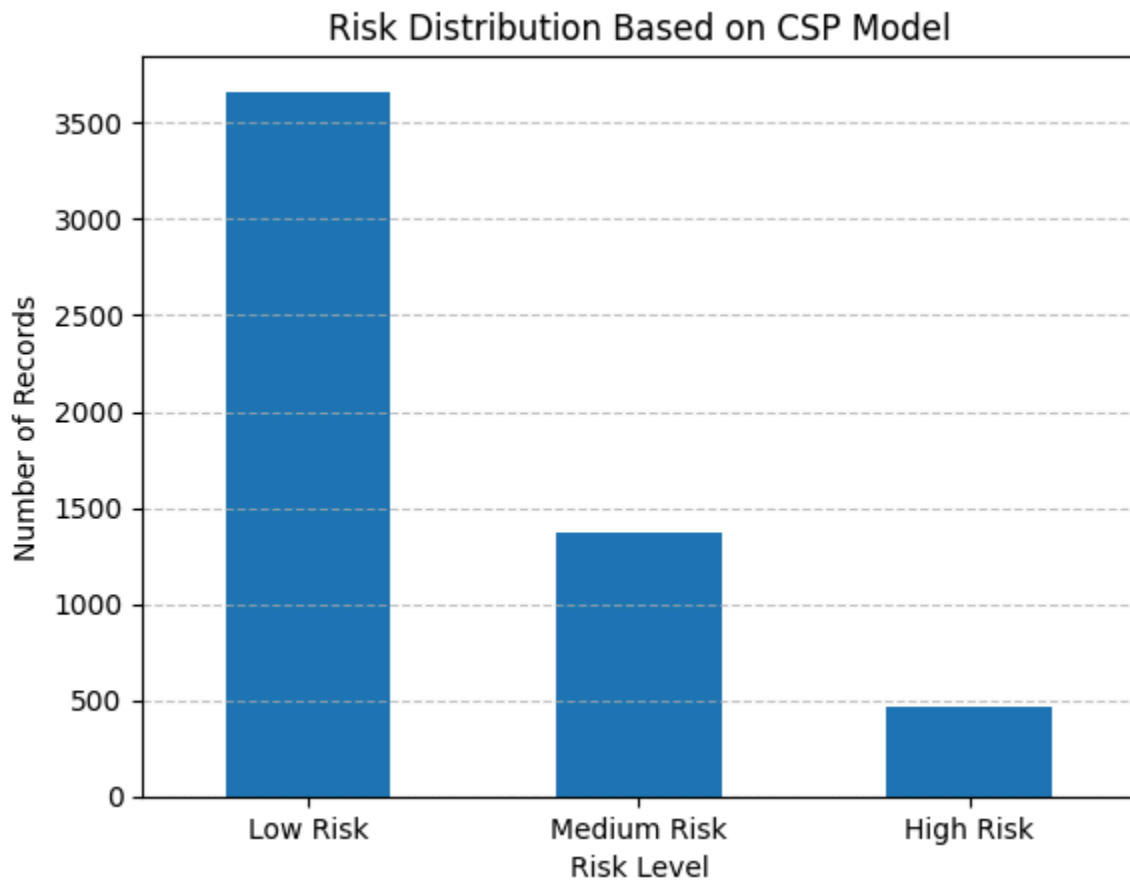
# Rename for better display (optional but recommended)
risk_counts = risk_counts.reindex(['Low Risk', 'Medium Risk', 'High Risk'])

risk_counts.plot(kind='bar')

plt.title("Risk Distribution Based on CSP Model")
plt.xlabel("Risk Level")
plt.ylabel("Number of Records")
plt.xticks(rotation=0)
plt.grid(axis='y', linestyle='--', alpha=0.7)

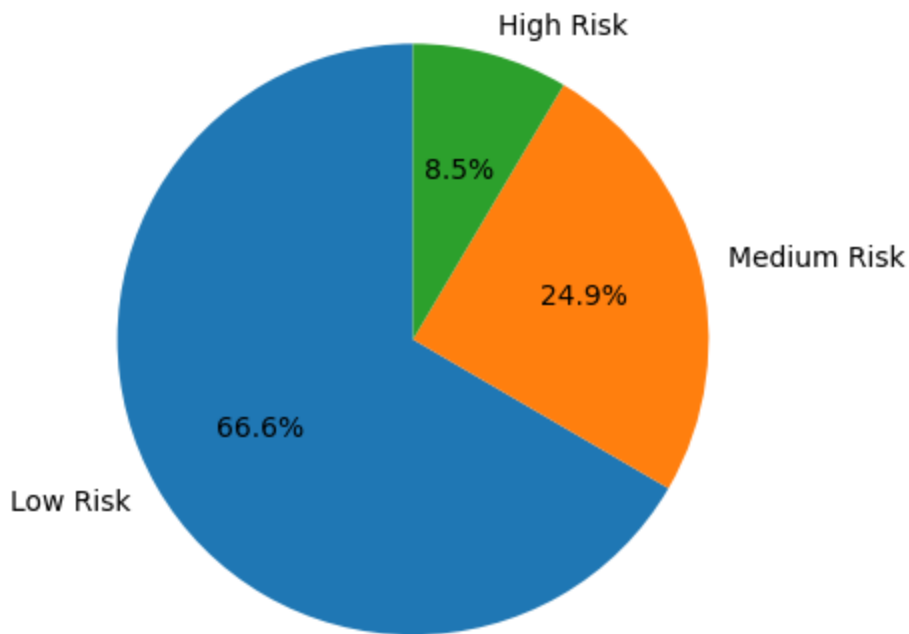
```

```
plt.show()
```



```
In [8]: # =====  
# PIE CHART (UPDATED)  
# =====  
import matplotlib.pyplot as plt  
  
risk_counts = df['AI_Result'].value_counts()  
  
# Ensure proper order  
risk_counts = risk_counts.reindex(['Low Risk', 'Medium Risk', 'High Risk'])  
  
risk_counts.plot(  
    kind='pie',  
    autopct='%1.1f%%',  
    startangle=90  
)  
  
plt.title("Risk Distribution Based on CSP Model")  
plt.ylabel("") # remove y-label  
plt.show()
```

Risk Distribution Based on CSP Model



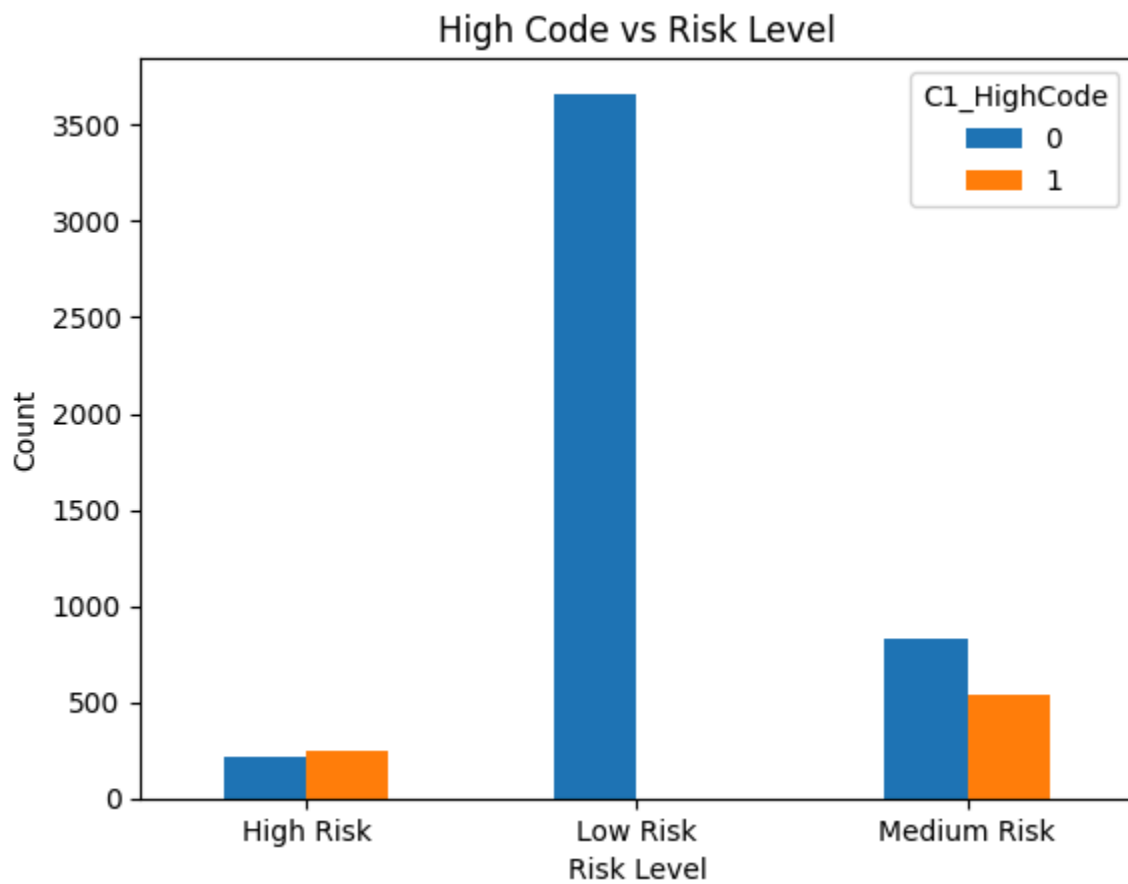
```
In [9]: # =====  
# CONSTRAINT vs RISK (UPDATED)  
# =====  
import matplotlib.pyplot as plt  
import pandas as pd  
  
# C1: High Code  
pd.crosstab(df['AI_Result'], df['C1_HighCode']).plot(kind='bar')  
plt.title("High Code vs Risk Level")  
plt.xlabel("Risk Level")  
plt.ylabel("Count")  
plt.xticks(rotation=0)  
plt.show()  
  
# C2: High Value  
pd.crosstab(df['AI_Result'], df['C2_HighValue']).plot(kind='bar')  
plt.title("High Value vs Risk Level")  
plt.xlabel("Risk Level")  
plt.ylabel("Count")  
plt.xticks(rotation=0)  
plt.show()  
  
# C3: Frequent Visits  
pd.crosstab(df['AI_Result'], df['C3_FrequentVisit']).plot(kind='bar')  
plt.title("Frequent Visits vs Risk Level")  
plt.xlabel("Risk Level")
```

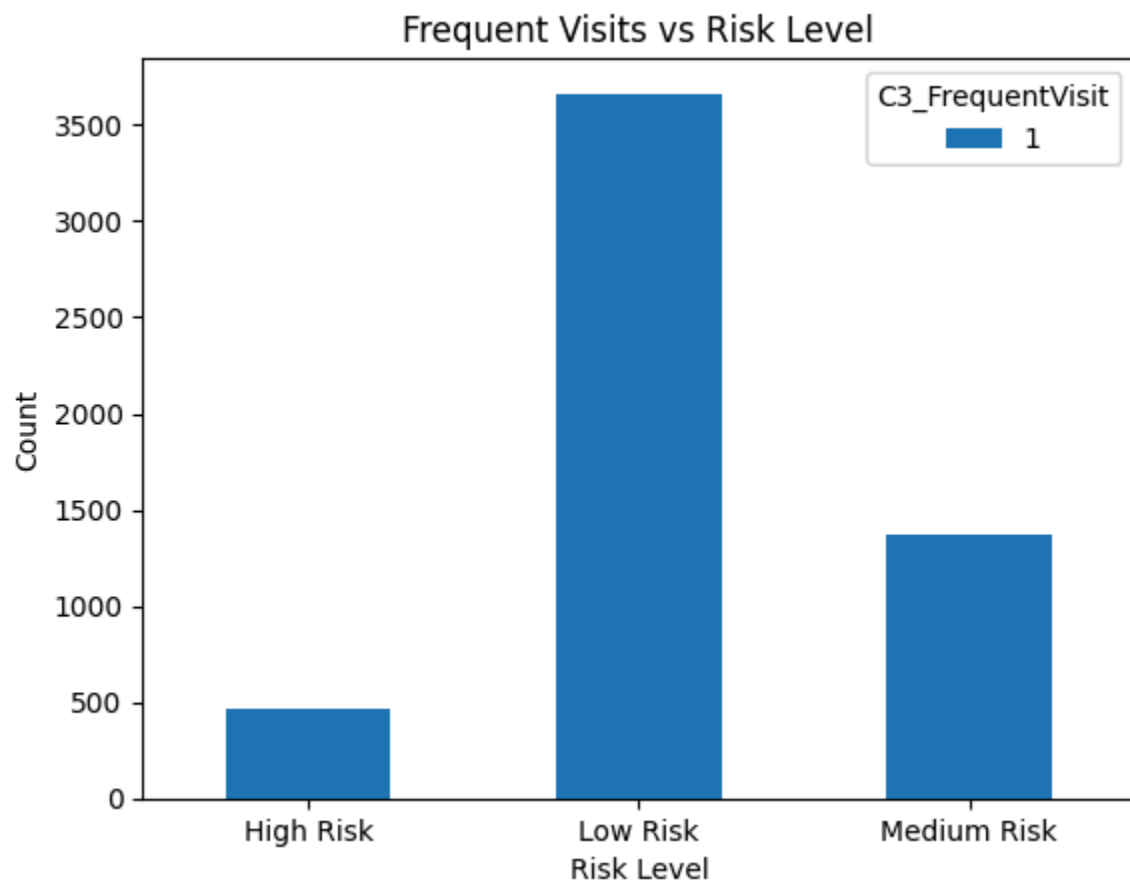
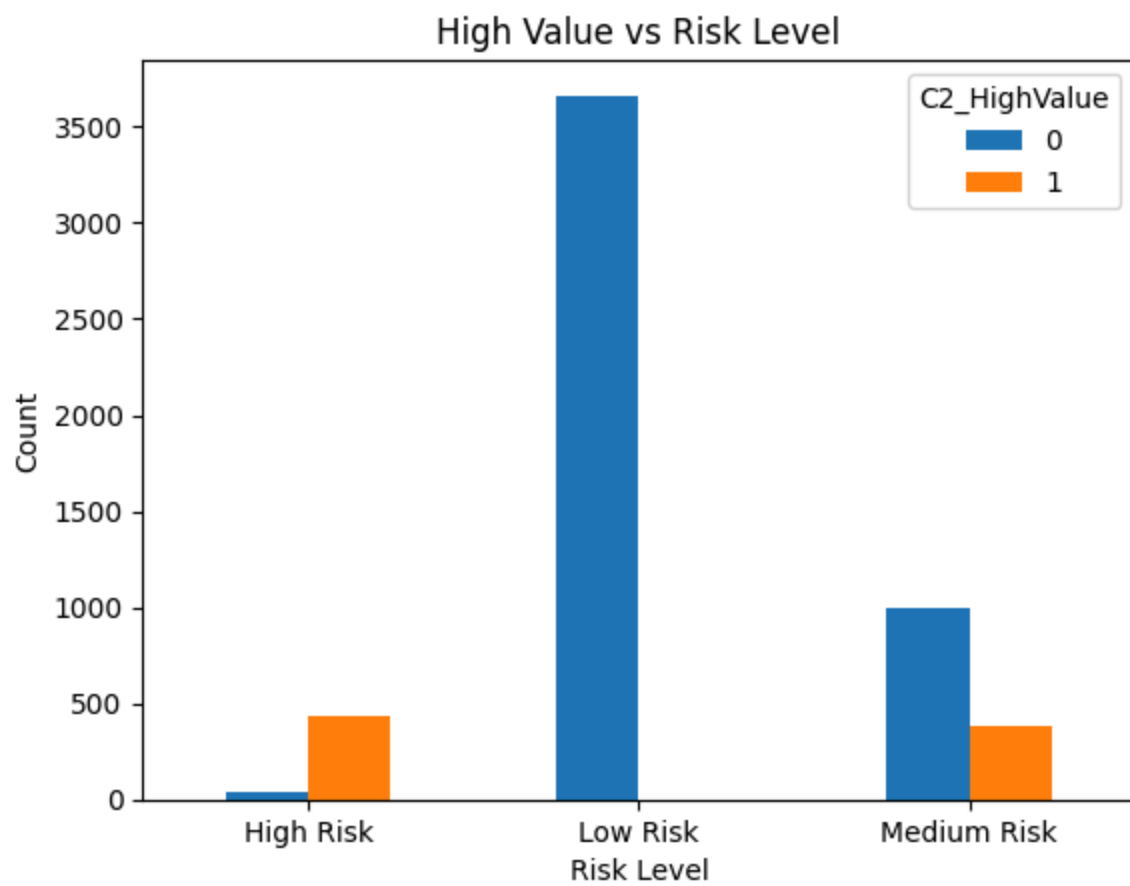
```

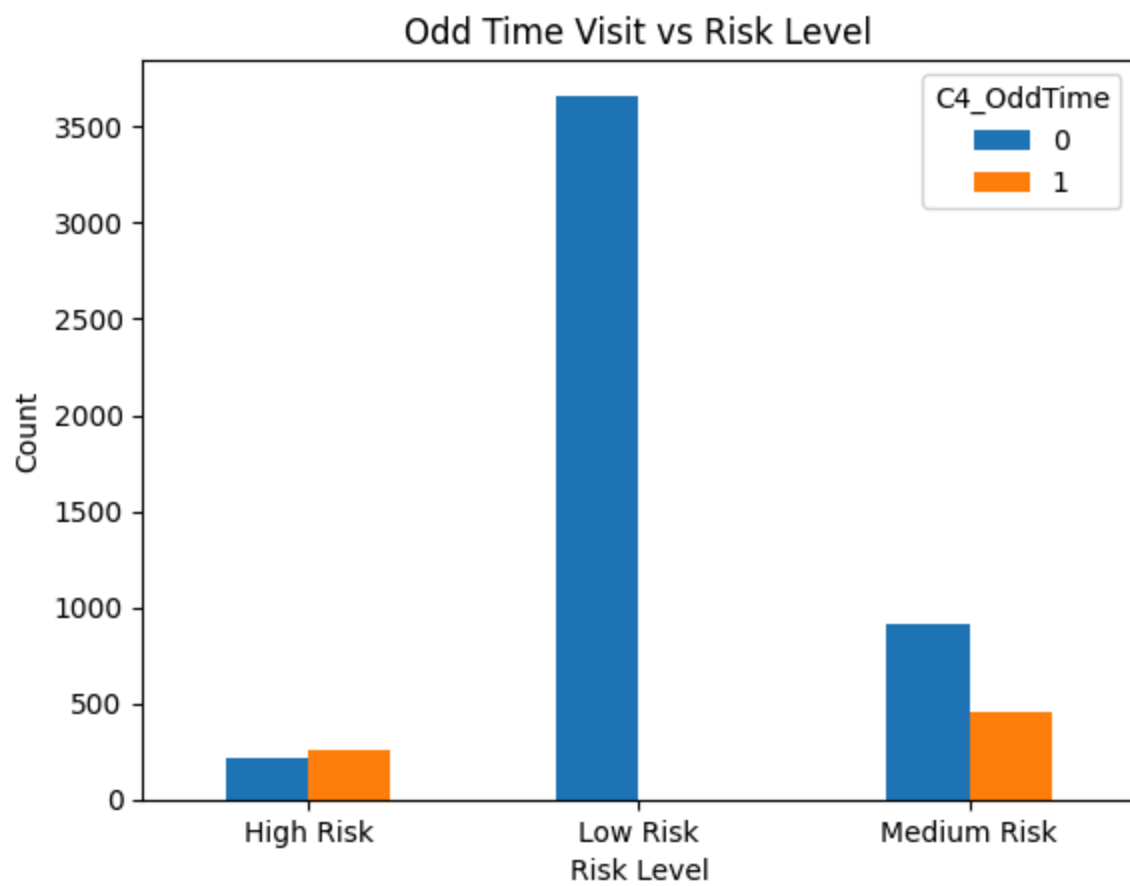
plt.ylabel("Count")
plt.xticks(rotation=0)
plt.show()

# C4: Odd Time
pd.crosstab(df['AI_Result'], df['C4_OddTime']).plot(kind='bar')
plt.title("Odd Time Visit vs Risk Level")
plt.xlabel("Risk Level")
plt.ylabel("Count")
plt.xticks(rotation=0)
plt.show()

```







```
In [10]: !pip install seaborn
```

Requirement already satisfied: seaborn in c:\users\asus\appdata\local\programs\python\python37\lib\site-packages (0.12.2)

Requirement already satisfied: numpy!=1.24.0,>=1.17 in c:\users\asus\appdata\local\programs\python\python37\lib\site-packages (from seaborn) (1.18.5)

Requirement already satisfied: pandas>=0.25 in c:\users\asus\appdata\local\programs\python\python37\lib\site-packages (from seaborn) (1.3.5)

Requirement already satisfied: matplotlib!=3.6.1,>=3.1 in c:\users\asus\appdata\local\programs\python\python37\lib\site-packages (from seaborn) (3.5.3)

Requirement already satisfied: typing_extensions in c:\users\asus\appdata\local\programs\python\python37\lib\site-packages (from seaborn) (4.7.1)

Requirement already satisfied: cycler>=0.10 in c:\users\asus\appdata\local\programs\python\python37\lib\site-packages (from matplotlib!=3.6.1,>=3.1->seaborn) (0.11.0)

Requirement already satisfied: fonttools>=4.22.0 in c:\users\asus\appdata\local\programs\python\python37\lib\site-packages (from matplotlib!=3.6.1,>=3.1->seaborn) (4.38.0)

Requirement already satisfied: kiwisolver>=1.0.1 in c:\users\asus\appdata\local\programs\python\python37\lib\site-packages (from matplotlib!=3.6.1,>=3.1->seaborn) (1.4.5)

Requirement already satisfied: packaging>=20.0 in c:\users\asus\appdata\local\programs\python\python37\lib\site-packages (from matplotlib!=3.6.1,>=3.1->seaborn) (24.0)

Requirement already satisfied: pillow>=6.2.0 in c:\users\asus\appdata\local\programs\python\python37\lib\site-packages (from matplotlib!=3.6.1,>=3.1->seaborn) (9.5.0)

Requirement already satisfied: pyparsing>=2.2.1 in c:\users\asus\appdata\local\programs\python\python37\lib\site-packages (from matplotlib!=3.6.1,>=3.1->seaborn) (3.1.4)

Requirement already satisfied: python-dateutil>=2.7 in c:\users\asus\appdata\local\programs\python\python37\lib\site-packages (from matplotlib!=3.6.1,>=3.1->seaborn) (2.9.0.post0)

Requirement already satisfied: pytz>=2017.3 in c:\users\asus\appdata\local\programs\python\python37\lib\site-packages (from pandas>=0.25->seaborn) (2025.1)

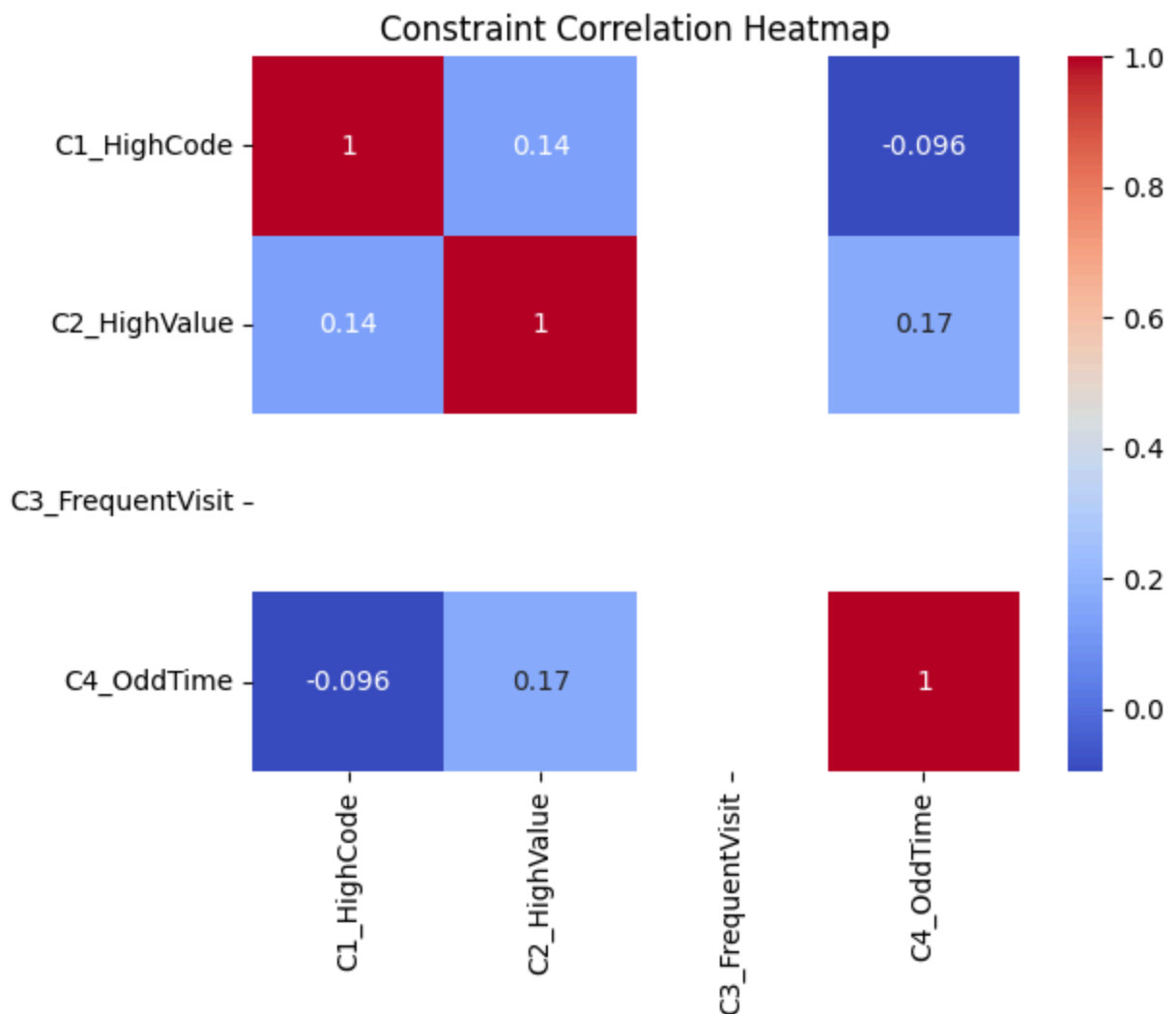
Requirement already satisfied: six>=1.5 in c:\users\asus\appdata\local\programs\python\python37\lib\site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.1->seaborn) (1.17.0)

```
In [11]: # =====
# HEATMAP (UPDATED)
# =====
import seaborn as sns
import matplotlib.pyplot as plt

corr_matrix = df[['C1_HighCode', 'C2_HighValue', 'C3_FrequentVisit', 'C4_OddTime']]

sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')

plt.title("Constraint Correlation Heatmap")
plt.show()
```



Before applying AI - The dataset only contained raw financial transaction data without any structured method to identify risky or anomalous behavior. There was no clear way to understand which transactions were safe, suspicious, or highly risky. Why AI was applied? - From base concept Problem: Fraud/anomaly is not directly visible. Requires evaluation of multiple financial conditions. Needs a systematic decision-making approach. 🙌 AI technique was applied to: Evaluate multiple risk conditions simultaneously. Identify abnormal financial behavior. Convert raw data into actionable insights. My Goal - Convert raw data → structured decision system. That's why I used: 🙌 CSP - Constraint Satisfaction Problem. AFTER APPLYING AI - What the system does now? For each transaction: Applies predefined constraints. Converts each condition into binary (0/1). Identifies which constraints are violated. Calculates total risk score. Assigns final risk category. Output created - Constraint Columns (C1-C4) → show which rules are violated. Risk Score (0-4) → number of violated constraints. AI_Result: High Risk → strong anomaly. Medium Risk → moderate suspicion. Low Risk → normal behavior. Comparison Before AI After AI Raw data Structured decision system. No clear interpretation. Constraint-based evaluation. Manual checking. Automated rule checking. No anomaly detection. Risk-based classification. FINAL INTERPRETATION. After applying CSP, the system was able to identify anomalous financial behavior by evaluating multiple predefined constraints. Each transaction was analyzed based on constraint violations, and risk levels were assigned accordingly. The inclusion of constraint columns (C1-C4) improves transparency, making the system explainable and easy to interpret. This demonstrates how CSP effectively transforms raw financial data into a structured and interpretable risk detection system.

```
In [12]: import pandas as pd

# =====
# LOAD DATA
```

```

# =====
df = pd.read_excel("d.csv.xlsx")

# Convert datetime
df['datetime'] = pd.to_datetime(df['datetime'])

risk_scores = []
results = []
reasons = []

# Patient frequency
patient_counts = df['patient_id'].value_counts()

for _, row in df.iterrows():

    risk = 0
    reason = []

    # =====
    # 🟡 CODE (IMPORTANT SIGNAL)
    # =====
    if row['code'] > 80:
        risk += 3
        reason.append("Very High Code")

    elif row['code'] > 60:
        risk += 2
        reason.append("High Code")

    elif row['code'] > 40:
        risk += 1
        reason.append("Moderate Code")

    # =====
    # 🔵 VALUE (MEASUREMENT)
    # =====
    if row['value'] > 2:
        risk += 3
        reason.append("Very High Value")

    elif row['value'] > 1:
        risk += 2
        reason.append("High Value")

    elif row['value'] > 0.5:
        risk += 1
        reason.append("Moderate Value")

    # =====
    # 🔵 FREQUENT VISITS
    # =====

```

```

if patient_counts[row['patient_id']] > 10:
    risk += 2
    reason.append("Very Frequent Visits")

elif patient_counts[row['patient_id']] > 5:
    risk += 1
    reason.append("Frequent Visits")

# =====
# 🕒 TIME ANALYSIS
# =====
hour = row['datetime'].hour

if hour < 5 or hour > 22:
    risk += 2
    reason.append("Critical Time")

elif hour < 7 or hour > 20:
    risk += 1
    reason.append("Odd Time")

# =====
# 📊 VALUE vs CODE RATIO (SMART RULE 🔥)
# =====
if row['code'] > 0:
    ratio = row['value'] / row['code']

    if ratio > 0.05:
        risk += 2
        reason.append("Abnormal Ratio")

    elif ratio > 0.02:
        risk += 1
        reason.append("Slightly High Ratio")

# =====
# FINAL SCORE
# =====
risk_scores.append(risk)

if risk >= 7:
    label = "High Risk"
elif risk >= 4:
    label = "Medium Risk"
else:
    label = "Low Risk"

results.append(label)

if len(reason) == 0:

```

```

        reasons.append("Normal Behavior")
    else:
        reasons.append(", ".join(reason))

# =====
# STORE RESULTS
# =====
df['Risk_Score'] = risk_scores
df['AI_Result'] = results
df['Reason'] = reasons

# =====
# OUTPUT
# =====
print("\n===== FINAL TABLE =====")
print(df[['datetime', 'patient_id', 'code', 'value',
          'Risk_Score', 'AI_Result', 'Reason']].head(10))

print("\n===== RISK DISTRIBUTION =====")
print(df['AI_Result'].value_counts())

# =====
# ACCURACY (ONLY IF TARGET EXISTS)
# =====
if 'target' in df.columns:
    from sklearn.metrics import accuracy_score

    df['Predicted'] = df['AI_Result'].apply(
        lambda x: 1 if x == "High Risk" else 0
    )

    accuracy = accuracy_score(df['target'], df['Predicted'])

    print("\n===== MODEL ACCURACY =====")
    print("Accuracy:", round(accuracy, 4))
else:
    print("\nNo target column → Accuracy not calculated")

# =====
# SAVE
# =====
df.to_csv("advanced_csp_output.csv", index=False)

print("\nSaved as advanced_csp_output.csv")

```

===== FINAL TABLE =====

	datetime	patient_id	code	value	Risk_Score	AI_Result \
0	1988-03-27 08:00:00	68	58	1	4	Medium Risk
1	1988-03-27 08:00:00	68	34	0	2	Low Risk
2	1988-03-27 12:00:00	68	60	1	4	Medium Risk
3	1988-03-27 18:00:00	68	62	2	7	High Risk
4	1988-03-28 08:00:00	68	58	1	4	Medium Risk
5	1988-03-28 08:00:00	68	34	0	2	Low Risk
6	1988-03-28 12:00:00	68	60	1	4	Medium Risk
7	1988-03-28 18:00:00	68	62	1	5	Medium Risk
8	1988-03-31 08:00:00	68	58	0	3	Low Risk
9	1988-03-31 08:00:00	68	34	0	2	Low Risk

Reason

0	Moderate Code, Moderate Value, Very Frequent V...
1	Very Frequent Visits
2	Moderate Code, Moderate Value, Very Frequent V...
3	High Code, High Value, Very Frequent Visits, S...
4	Moderate Code, Moderate Value, Very Frequent V...
5	Very Frequent Visits
6	Moderate Code, Moderate Value, Very Frequent V...
7	High Code, Moderate Value, Very Frequent Visits
8	Moderate Code, Very Frequent Visits
9	Very Frequent Visits

===== RISK DISTRIBUTION =====

Low Risk 2949
Medium Risk 1948
High Risk 603
Name: AI_Result, dtype: int64

No target column → Accuracy not calculated

Saved as advanced_csp_output.csv

In []: